

AI-TECH

Przykład 4 DO LABORATORIUM 1 z UCZENIA MASZYNOWEGO

Regresja liniowa i logistyczna

Tomasz Neumann

Regresja logistyczna w scikit-learn

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020,
Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2
"Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia
Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

1. Przykład 4

Regresja logistyczna służy do szacowania prawdopodobieństwa przynależności przykładu do danej klasy. W przypadku oszacowania prawdopodobieństwa większego od 50%, model może prognozować, że przykład należy do tej klasy (tzw. klasy pozytywnej) - w przeciwnym razie zostaje przypisany do tzw. klasy negatywnej. Jak nie trudno zauważyć, model taki wykazuje cechy klasyfikatora binarnego. W przypadku regresji logistycznej także należy wyliczyć ważoną sumę cech wejściowych, jednak rezultatem jest funkcja logistyczna, a nie bezpośrednio wyniki. Postać wektorową modelu logistycznego można zapisać jako

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\mathbf{x}^T \boldsymbol{\theta})$$

przy czym funkcja logistyczna w postaci $\sigma(\cdot)$ jest funkcją sigmoidalną

$$\sigma(t) = \frac{1}{1 + e^{-t}},$$

której wartości są z przedziału od 0 do 1. Poniżej został przedstawiany wykres funkcji logistycznej. Na początku, tak jak zwykle zaimportujemy podstawowe biblioteki do naszego projektu

```
In [1]: # Import podstawowych bibliotek języka Python
import numpy as np # operacje numeryczne
import matplotlib # wykresy i ich ustawienia
import matplotlib.pyplot as plt # tworzenie wykresu
```

oraz stworzony zostanie zbiór pięciuset danych t z przedziału od -10 do 10. Po dokonaniu odpowiednich ustawień wykresu oraz wykreśleniu wygenerowanych danych wraz z liniami pomocniczymi, wyświetli się wykres funkcji sigmoidalnej, której wartości są bardzo bliskie zera lub jedności w zakresie swojej dziedziny, co sprawia, że nadaje się do stosowania w metodach klasyfikacyjnych.

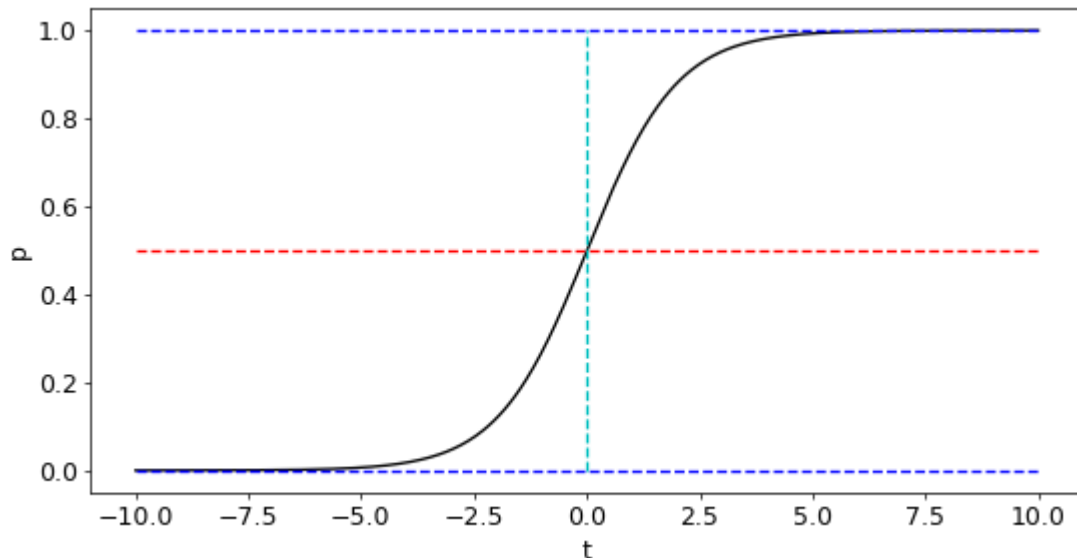
```
In [2]: dataset_training_size = 500
t = x_t = np.linspace(-10, 10, dataset_training_size).astype(np.float32)
sigma = (1+np.exp(-t))**(-1)

# Wizualizacja wygenerowanych danych
# Ustawienia związane z wyświetlaniem
matplotlib.rcParams.update({'font.size': 14})
f = plt.figure(figsize=(10, 5), dpi=64)
f1 = f.add_subplot(111)
```

```

f1.set_xlabel('t')
f1.set_ylabel('p')
f1.plot(t,sigma, "k")
#wykreślenie linii pomocniczych
f1.plot(t,0*t+1, "b--")
f1.plot(t,0*t, "b--")
f1.plot(t,0*t+0.5, "r--")
f1.plot(0*t,sigma, "c--")
plt.show()

```



Na podstawie powyższego wykresu, można łatwo obliczyć prognozę, gdyż model regresji logistycznej przewiduje 1, jeżeli wynik $\mathbf{x}^T \boldsymbol{\theta}$ jest dodatni ($t \geq 0$) natomiast 0, jeżeli jest ujemny ($t < 0$). Poniżej został przedstawiony kod, umożliwiający dokonać klasyfikacji binarnej z jedną zmienną (zbiór cyfr) do dwóch klas.

Dodatkowo do projektu zaimportujemy pakiet scikit-learn, w którym jest już zaimplementowany algorytm regresji logistycznej oraz wiele innych użytecznych narzędzi, które będziemy wykorzystywać w trakcie tego przykładu.

```

In [5]: # Import dodatkowej biblioteki scikit-learn
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

```

Na potrzeby tego przykładu stworzymy tablice NumPy równomiernie rozłożonych wartości wejściowych x i wyjściowych y . Tablica x musi być dwuwymiarowa, w której powinna być jedna kolumna dla każdego wejścia, a liczba wierszy powinna być równa liczbie obserwacji. Tablica y jest jednowymiarową tablicą z 10-cioma elementami.

```

In [6]: # Stworzenie przykładowych danych

# Dwuwymiarowa tablica danych wejściowych:
# jedna kolumna dla jednego wejścia, liczba wierszy równa liczbie obserwacji
x = np.arange(10).reshape(-1, 1)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1])
#wyświetlenie kontrolne utworzonych danych
print(x)
print(y)

```

```
[[0]
 [1]
 [2]
 [3]
 [4]
 [5]
 [6]
 [7]
 [8]
 [9]]
[0 0 0 0 1 1 1 1 1 1]
```

Zbudowanie modelu klasyfikacji nastąpi za pomocą klasy `LogisticRegression`. Klasa ta ma kilka opcjonalnych parametrów, które definiują zachowanie modelu - więcej informacji znajdziesz na stronie [https://scikit-](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html)

[learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html).

Trening modelu następuje po wywołaniu metody `fit` na tak spreparowanych wcześniej danych.

```
In [7]: # Zbudowanie modelu klasyfikacji za pomocą klasy LogisticRegression
# https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Logisti
model = LogisticRegression(solver='liblinear', random_state=0)
# wytrenowanie modelu na podstawie wcześniej wygenerowanych danych
model.fit(x, y)
```

```
Out[7]: LogisticRegression(random_state=0, solver='liblinear')
```

Informacje z dokumentacji na temat **`sklearn.linear_model.LogisticRegression`**

```
class sklearn.linear_model. LogisticRegression (penalty='l2', *, dual=False,
tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None,
random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0,
warm_start=False, n_jobs=None, l1_ratio=None)
```

Należy zauważyć, że wykorzystane wyżej wywołanie inicjujące model regresji logistycznej poprzez wybór metody mającej doprowadzić do dopasowania modelu do danych wiąże się także z ustawieniami wielu innych parametrów wywołania, które są ustawione standardowo domyślnymi wartościami. Przy tak zdefiniowanym modelu można także sprawdzać właściwości stworzonego modelu. Atrybut `.classes_` wywołany na stworzonym modelu wyświetli tablicę indywidualnych wartości znajdujących się w `y`. Można także pobrać wartości dla modelu liniowego za pomocą `.intercept_` oraz `model.coef_`.

```
In [8]: # Wyświetlenie informacji o modelu

# wyświetlenie wartości klas y
print('Klasy występujące w modelu: ', model.classes_)
# parametry prostej, opisujące nachylenie krzywej podziału na klasy
print('a0: ', model.intercept_)
print('a1: ', model.coef_)
```

```
Klasy występujące w modelu: [0 1]
a0: [-1.04608067]
a1: [[0.51491375]]
```

Dla tak zdefiniowanego modelu za pomocą `.predict_proba()` można zwrócić macierz 'prawdopodobieństwa', w której pierwsza kolumna w tym przypadku informuje o szansie

przynależności do klasy 0, a druga kolumna informuje o szansie przynależności do klasy 1 pojedynczej obserwacji. Sprawdzenie predykcji próbek wejściowych do określonych klas można uzyskać korzystając z `.predict()`. Z kolei użycie `.score()` daje nam informację o dokładności naszego modelu.

```
In [9]: # predict_proba(WEJŚCIE) - zwrócenie macierzy prawdopodobieństw
# że na wyjściu uzyskamy przypisanie do klasy 0 lub 1
print("Macierz 'prawdopodobieństwa'/szans przynależności do odpowiednich klas dla")
print(model.predict_proba(x))
# predict - zwracana przewidziane wartości w tablicy 1D
print('\nPrzewidziana przez model przynależność do klas zmiennej wejściowej')
print(model.predict(x))
#score - zwraca stosunek poprawnych predykcji do wszystkich obserwacji
print('\nZwrócenie dokładności predykcji')
print(model.score(x, y))
```

Macierz 'prawdopodobieństwa'/szans przynależności do odpowiednich klas dla danych wejściowych

```
[[0.74002157 0.25997843]
 [0.62975524 0.37024476]
 [0.5040632  0.4959368 ]
 [0.37785549 0.62214451]
 [0.26628093 0.73371907]
 [0.17821501 0.82178499]
 [0.11472079 0.88527921]
 [0.07186982 0.92813018]
 [0.04422513 0.95577487]
 [0.02690569 0.97309431]]
```

Przewidziana przez model przynależność do klas zmiennej wejściowej

```
[0 0 0 1 1 1 1 1 1 1]
```

Zwrócenie dokładności predykcji

```
0.9
```

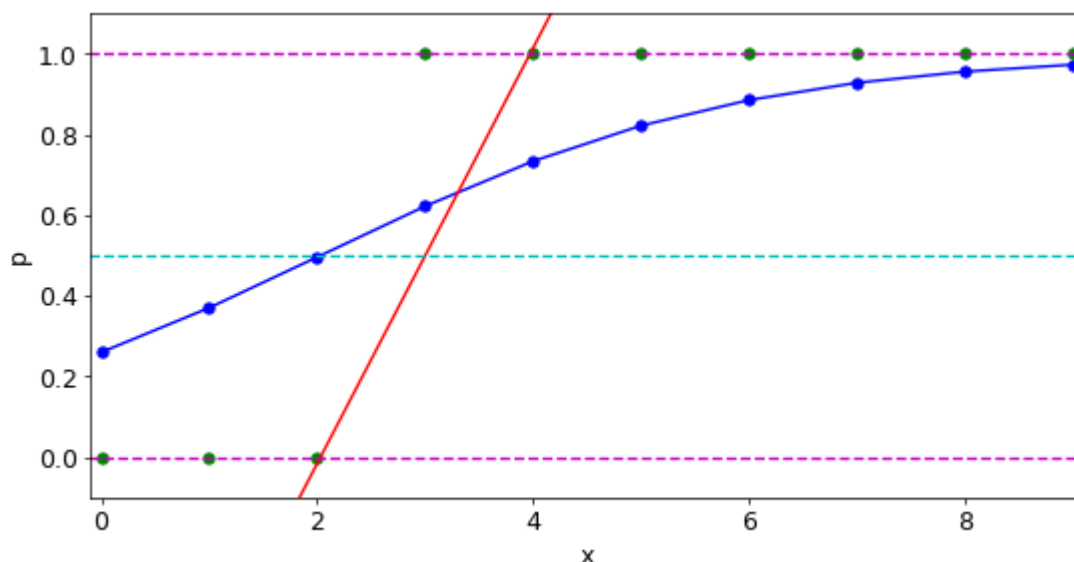
Powyższe wartości można w łatwy sposób zinterpretować i zilustrować za pomocą biblioteki matplotlib, co zostało przedstawione w poniższych dwóch sekcjach.

```
In [10]: prob = (model.predict_proba(x)).reshape(20,)[1::2]
print(prob)
print(y.shape, prob.shape)
```

```
[0.25997843 0.37024476 0.4959368  0.62214451 0.73371907 0.82178499
 0.88527921 0.92813018 0.95577487 0.97309431]
(10,) (10,)
```

```
In [11]: # Wizualizacja wygenerowanych danych
# Ustawienia związane z wyświetlaniem
matplotlib.rcParams.update({'font.size': 14})
f = plt.figure(figsize=(10, 5), dpi=64)
f1 = f.add_subplot(111)
f1.set_xlabel('x')
f1.set_ylabel('p')
f1.set_xlim([-0.1, 9.1])
f1.set_ylim([-0.1, 1.1])
# wartości prawdopodobieństwa przypisania do klasy 1:
prob = ((model.predict_proba(x).reshape(20,))[1::2])
f1.plot(x, prob, "bo-")
```

```
# wynik predykcji
f1.plot(x,model.predict(x), 'go')
# wykreślenie zależności z współczynników liniowych
y_lin = model.intercept_ + model.coef_*x
f1.plot(x,y_lin, 'r')
#wykreślenie linii pomocniczych
f1.plot(t,0*t+1, "m--")
f1.plot(t,0*t, "m--")
f1.plot(t,0*t+0.5, "c--")
plt.show()
```



W celu uzyskania większej ilości informacji na temat dokładności modelu, można skorzystać z macierzy pomyłek, co w tym przypadku sprowadza się do użycia funkcji `confusion_matrix()` działającej na aktualnych danych oraz tych uzyskanych z predykcji.

```
In [12]: confusion_matrix(y, model.predict(x))
```

```
Out[12]: array([[3, 1],
               [0, 6]], dtype=int64)
```

W rezultacie otrzymamy się informacje o predykcjach prawdziwie negatywnych TN (3), fałszywie negatywnych FN (0), fałszywych pozytywnych FP (1) oraz o prawdziwych pozytywnych TP (6).

Pełniejszy raport z procesu klasyfikacji uzyskuje się za pomocą funkcji `classification_report()`, gdzie jako argumenty podaje się również aktualne dane oraz te uzyskane z predykcji.

```
In [13]: print(classification_report(y, model.predict(x)))
```

	precision	recall	f1-score	support
0	1.00	0.75	0.86	4
1	0.86	1.00	0.92	6
accuracy			0.90	10
macro avg	0.93	0.88	0.89	10
weighted avg	0.91	0.90	0.90	10

Zadania do wykonania

Zadanie 1

Zbadaj zachowanie się treningu modelu w zależności od różnych wartości parametru C , który określa siłę regularyzacji.

Zadanie 2

Stwórz dane wejściowe o liczbie większej niż 100, które nie będą reprezentowały równych klas np. w stosunku 3:7. Jaki będzie wynik predykcji takiego modelu.



Rzeczpospolita
Polska

Unia Europejska
Europejski Fundusz
Rozwoju Regionalnego

